Polar Codes for the Two-User Multiple Access Channel

Design, Decoding, and Neural Decoder Approaches

Project Documentation

March 2026

Abstract

This document provides a comprehensive description of a software library for polar codes on the two-user binary-input multiple access channel (MAC). The project supports three channel models—Binary Erasure MAC (BEMAC), Additive Binary Noise MAC (ABNMAC), and Gaussian MAC—with both analytical and Monte Carlo code design. Five decoder implementations are provided, ranging from an $O(N^2)$ reference decoder to an $O(N \log N)$ computational graph decoder supporting all monotone chain decoding paths, a Numba-JIT parallel decoder, and an SC List (SCL) decoder. The project also includes a neural decoder based on the computational graph framework, using learned MLP operations with a Soft-Bit Bridge for the calcParent step. Extensive simulation results (BLER vs. block length, SNR sweeps, rate analysis) are presented for all channel models and code classes.

Contents

1 Executive Summary **3**
- 1.1 Key Contributions . . . 3
- 1.2 Key Findings . . . 3

2 Background and Theory **3**
- 2.1 Polar Codes (Arikan 2009) . . . 3
- 2.2 Multiple Access Channel (MAC) Model . . . 4
- 2.3 Channel Models . . . 4
  - 2.3.1 Binary Erasure MAC (BEMAC) . . . 4
  - 2.3.2 Additive Binary Noise MAC (ABNMAC) . . . 4
  - 2.3.3 Gaussian MAC . . . 5
- 2.4 Code Classes and Decoding Paths . . . 5

3 Code Design **5**
- 3.1 Analytical Bhattacharyya Design (`design.py`) . . . 5
- 3.2 Monte Carlo Genie-Aided Design (`design_mc.py`) . . . 6
- 3.3 Gaussian MAC Design (`design_mc.py`) . . . 6
- 3.4 Rate Selection . . . 7
- 3.5 Design File Inventory . . . 7

4 Encoder (`encoder.py`) **7**
- 4.1 Implementation . . . 7
- 4.2 Batch and TensorFlow Variants . . . 7

5 Decoders **8**
- 5.1 Overview . . . 8
- 5.2 LLR-Based SC Decoder (Extreme Paths) . . . 8
- 5.3 Computational Graph Decoder (All Paths) . . . 8
  - 5.3.1 Tree Structure and Indexing . . . 9
  - 5.3.2 Core Operations . . . 9
  - 5.3.3 Navigation and decodeAt Flow . . . 9
  - 5.3.4 DecodeAt Algorithm . . . 10
  - 5.3.5 Root Initialization . . . 10
- 5.4 Numba Parallel Decoder (`decoder_parallel.py`) . . . 10
- 5.5 SC List Decoder (`decoder_scl.py`) . . . 11
- 5.6 Batch Vectorization . . . 11

6 Neural Decoders **11**
- 6.1 Neural Computational Graph (NCG) Architecture . . . 11
  - 6.1.1 Architecture Components . . . 11
  - 6.1.2 Configuration . . . 12
- 6.2 Memory Channel Extension (`ncg_memory.py`) . . . 12
- 6.3 Training Procedure (`neural/train.py`) . . . 12
- 6.4 Saved Models . . . 12

7 Evaluation Framework **13**
- 7.1 MACEval Class (`eval.py`) . . . 13
- 7.2 Simulation Scripts . . . 13

8 Results **13**
8.1 BEMAC Results: BLER vs. Block Length . . . 14
8.1.1 Class C (Path $0^N 1^N$) . . . 14
8.2 BEMAC Class B Results . . . 14
8.3 Gaussian MAC Results . . . 14
8.3.1 SC Decoder Results ($L = 1$) . . . 15
8.3.2 SCL Decoder Results ($L = 32$) . . . 15
8.3.3 Key Observations . . . 15
8.4 ABNMAC Results . . . 16
8.4.1 ABNMAC Design Considerations . . . 16
8.5 SC vs. SCL Comparison on BEMAC . . . 16
8.6 Results Directory Structure . . . 16
8.7 Generated Plots . . . 17
8.8 NCG Neural Decoder Results . . . 17

9 Implementation Notes **17**
9.1 Indexing Conventions . . . 17
9.2 Log-Domain Arithmetic . . . 18
9.3 Numerical Stability . . . 18
9.4 Performance Optimization . . . 18
9.5 Numba Closure Bug . . . 19
9.6 Design Comparison: Analytical vs. Monte Carlo . . . 19

10 Scripts Reference **19**

11 File Structure and API Reference **19**
11.1 Project File Tree . . . 19
11.2 Public API (`polar/__init__.py`) . . . 20

12 References **21**

1 Executive Summary

This project implements polar codes for the two-user binary-input multiple access channel (MAC), following the framework of Önay [2] and Ren, Nasser, and Bhatt [3].

1.1 Key Contributions

(1) **Three channel models:** BEMAC ($Z = X + Y$), ABNMAC ($Z = (X \oplus E_x, Y \oplus E_y)$), and Gaussian MAC ($Z = (1-2X) + (1-2Y) + W$).

(2) **Analytical and Monte Carlo code design:** Bhattacharyya-parameter recursion (Arikan 2009) for BEMAC and ABNMAC; genie-aided MC design for arbitrary channels and paths.

(3) **Five decoder backends:**

- Reference $O(N^2)$ SC decoder (`decoder.py`)
- Efficient $O(N \log N)$ LLR-based decoder for extreme paths (`efficient_decoder.py`)
- Interleaved $O(N \log N)$ computational graph decoder for all monotone chain paths (`decoder_interleaved.py`)
- Numba-JIT parallel decoder with batch `prange` parallelism (`decoder_parallel.py`)
- SCL decoder with list sizes up to $L = 32$ (`decoder_scl.py`)

(4) **Neural SC decoder (NCG):** MLP-based tree operations with a Soft-Bit Bridge for calcParent, matching or beating analytical SC for BEMAC Class B at $N = 8$–128.

(5) **Extensive simulation results:** BEMAC BLER vs. N for Classes A, B, C; ABNMAC across block lengths $N = 8$–1024; Gaussian MAC campaign with 824 data points (3 classes $\times$ multiple SNRs $\times$ $N = 8$–2048, SC and SCL).

1.2 Key Findings

- **BEMAC Class C** (path $0^N 1^N$) achieves BLER $\approx 2.6 \times 10^{-4}$ at $N = 1024$ with SC decoding at rate ratio $\rho = 0.6$.
- **BEMAC Class A** (symmetric rate direction) suffers from error floor: BLER $\to 1.0$ as N increases, confirming that the symmetric path is capacity-suboptimal for the asymmetric BEMAC capacity region.
- **Gaussian MAC:** SC performance is strongly SNR-dependent. At SNR = 6 dB and rate ratio $\rho = 0.5$, Classes A and B achieve BLER $\to 0$ as N grows, while $\rho = 0.9$ causes error floors.
- **NCG neural decoder:** At $N = 128$ on BEMAC Class B, the neural decoder achieves BLER = 0.0018 vs. SC BLER = 0.0034—a 47% improvement.

2 Background and Theory

2.1 Polar Codes (Arikan 2009)

Polar codes are a family of capacity-achieving codes based on the phenomenon of *channel polarization* [1]. Given a binary-input memoryless channel W, one constructs $N = 2^n$ synthetic sub-channels $W_N^{(i)}$ for $i = 1, \ldots, N$. As $N \to \infty$, each sub-channel polarizes to either a noiseless channel or a completely noisy channel.

The generator matrix is $G_N = B_N \cdot F^{\otimes n}$ where $F = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ is the Arikan kernel and B_N is the bit-reversal permutation matrix. Encoding is $x^N = u^N \cdot G_N$ over GF(2).

The Bhattacharyya parameter $Z(W) \in [0, 1]$ measures channel reliability. The Arikan recursion for polarization is:

$$Z(W^-) = 2Z(W) - Z(W)^2 \quad \text{(bad split, higher } Z\text{)} \tag{1}$$

$$Z(W^+) = Z(W)^2 \quad \text{(good split, lower } Z\text{)} \tag{2}$$

Information bits are placed on sub-channels with $Z(W_N^{(i)}) \approx 0$ (reliable); the remaining positions are set to known frozen values.

2.2 Multiple Access Channel (MAC) Model

The two-user binary-input MAC has inputs $X \in \{0, 1\}$ (User 1) and $Y \in \{0, 1\}$ (User 2), with channel transition probability $W(z|x, y)$. The capacity region under uniform input distributions is characterized by:

$$R_x \leq I(Z; X) \tag{3}$$

$$R_y \leq I(Z; Y|X) \tag{4}$$

$$R_x + R_y \leq I(Z; X, Y) \tag{5}$$

The polar coding approach for the MAC treats the two users' messages as two separate polar codes that share the same channel output z^N. Successive cancellation (SC) decoding alternates between decoding User 1 (U) bits and User 2 (V) bits according to a *decoding path* b^{2N}.

2.3 Channel Models

2.3.1 Binary Erasure MAC (BEMAC)

$$Z = X + Y \in \{0, 1, 2\} \tag{6}$$

This is a deterministic channel. The output $Z = 1$ is ambiguous (either $(X, Y) = (0, 1)$ or $(1, 0)$), while $Z = 0$ and $Z = 2$ uniquely determine both inputs. The capacity region (uniform inputs) is:

$$R_x \leq 0.5, \quad R_y \leq 0.5, \quad R_x + R_y \leq 1.0$$

The Bhattacharyya starting values for the extreme path $0^N 1^N$ are:

- $Z_0^{(U)} = 0.5$ (U marginal effective channel is BEC(0.5))
- $Z_0^{(V)} = 0.0$ (V conditional channel is noiseless once X is known)

2.3.2 Additive Binary Noise MAC (ABNMAC)

$$Z = (X \oplus E_x,\ Y \oplus E_y), \quad (E_x, E_y) \sim p_{\text{noise}} \tag{7}$$

The noise distribution is a 2×2 joint probability matrix. The default parameters (from Önay 2013, Table I) are:

$$p_{\text{noise}} = \begin{pmatrix} 0.1286 & 0.0175 \\ 0.0175 & 0.8364 \end{pmatrix}$$

yielding capacities $I(Z; X) \approx 0.400$, $I(Z; Y|X) \approx 0.800$, $I(Z; X, Y) \approx 1.200$ bits. The Bhattacharyya starting values are:

$$Z_0^{(U)} = 2\sqrt{p_{E_x}(0) \cdot p_{E_x}(1)} \approx 0.706 \tag{8}$$

$$Z_0^{(V)} = 2\left[\sqrt{p_{00}p_{01}} + \sqrt{p_{10}p_{11}}\right] \approx 0.337 \tag{9}$$

2.3.3 Gaussian MAC

$$Z = (1 - 2X) + (1 - 2Y) + W, \quad W \sim \mathcal{N}(0, \sigma^2) \tag{10}$$

where $X, Y \in \{0, 1\}$ are BPSK-modulated: $s_x = 1-2X \in \{-1, +1\}$. The per-user SNR is SNR $= 1/\sigma^2$. The noiseless constellation points are $\{-2, 0, 0, +2\}$ for $(X, Y) \in \{(1, 1), (0, 1), (1, 0), (0, 0)\}$.

The V-conditional Bhattacharyya parameter has a closed form: $Z_0^{(V)} = e^{-1/(2\sigma^2)}$, while the U-marginal parameter $Z_0^{(U)}$ requires numerical integration over the mixture Gaussian marginal channel.

2.4 Code Classes and Decoding Paths

The decoding path b^{2N} is a binary vector of length $2N$ that specifies the order in which U and V bits are decoded. A 0 in position k means "decode one U bit"; a 1 means "decode one V bit." The project uses the *monotone chain* path family:

$$b^{2N} = 0^{\pi}\, 1^{N}\, 0^{N-\pi}, \quad 0 \le \pi \le N \tag{11}$$

Key code classes are:

- **Class C** ($\pi = N$): $b = 0^N 1^N$ — decode all U bits first, then all V bits. Corresponds to R_U direction along the U-axis of the capacity region.
- **Class A** ($\pi = 0$): $b = 1^N 0^N$ — decode all V bits first, then all U bits.
- **Class B** ($\pi = N/2$): Intermediate path, interleaving U and V decoding.

For extreme paths (Classes A and C), the decoder decomposes into two sequential single-user SC decodes with $O(N \log N)$ complexity each. For intermediate paths, the computational graph approach [3] is required.

3 Code Design

3.1 Analytical Bhattacharyya Design (`design.py`)

The analytical design applies n stages of the Arikan recursion starting from a channel-specific initial Bhattacharyya parameter Z_0.

Algorithm 1 Bhattacharyya Recursion

Require: Starting parameter $Z_0 \in [0, 1]$, stages n

```
z ← [Z_0]
for stage = 1, ..., n do
    z_bad ← min(1, 2z − z^2)                    ▷ Z^− split
    z_good ← z^2                                ▷ Z^+ split
    Interleave: z_new[2k] = z_bad[k], z_new[2k+1] = z_good[k]
end for
return z of length N = 2^n
```

For each channel model, the starting parameters are computed analytically:

- **BEMAC:** $Z_0^{(U)} = 0.5$, $Z_0^{(V)} = 0.0$
- **ABNMAC:** $Z_0^{(U)} = 2\sqrt{p_{E_x}(0) \cdot p_{E_x}(1)}$, $Z_0^{(V)} = 2[\sqrt{p_{00}p_{01}} + \sqrt{p_{10}p_{11}}]$

The best k_u (resp. k_v) sub-channels with smallest Bhattacharyya parameters form the information sets A_U (resp. A_V). All positions use 1-based indexing throughout the codebase.

The function `design_bemac(n, ku, kv)` returns:

- A_U, A_V: sorted lists of 1-indexed information positions
- `frozen_u`, `frozen_v`: dictionaries mapping frozen positions to their values (typically 0)
- z_u, z_v: Bhattacharyya parameter arrays

3.2 Monte Carlo Genie-Aided Design (`design_mc.py`)

For channels where analytical Bhattacharyya computation is unavailable (e.g., Gaussian MAC) or for intermediate paths, the project provides a Monte Carlo design procedure:

1. Generate random message pairs (u, v)
2. Encode both users: $x = u \cdot G_N$, $y = v \cdot G_N$
3. Transmit through the channel to get z
4. Run a *genie-aided* SC decoder that always feeds the true bit value to subsequent steps, but records whether each bit would have been decoded incorrectly
5. Accumulate per-bit error counts over many trials
6. Rank sub-channels by error rate (ascending = most reliable first)

Tie-breaking uses bit-reversal weight: when multiple channels have identical MC error rates (common when $P_e = 0$ for many channels), the natural polar reliability ordering breaks ties via the bit-reversed index.

The design files are stored as NumPy `.npz` archives containing `sorted_u`, `sorted_v`, `error_rates_u`, `error_rates_v`, and optionally `path_i`.

3.3 Gaussian MAC Design (`design_mc.py`)

The Gaussian MAC presents a unique design challenge: the continuous output alphabet makes analytical Bhattacharyya computation more complex. While the V-conditional parameter has a closed-form $Z_0^{(V)} = e^{-1/(2\sigma^2)}$, the U-marginal parameter requires numerical integration over the mixture Gaussian channel.

For non-extreme paths (Classes A and B), the analytical Bhattacharyya approach does not directly apply because the effective channels depend on the decoding order. The MC design procedure handles this naturally by simulating the actual genie-aided SC decoder along the specified path.

The Gaussian MAC design workflow is:

1. Choose class (A, B, or C), block length $N = 2^n$, and SNR in dB
2. Create a `GaussianMAC` channel: $\sigma^2 = 10^{-\mathrm{SNR}_{\mathrm{dB}}/10}$
3. Run `mc_design(n, channel, mc_trials, path_i=pi)` to estimate per-bit error rates for both users
4. Select the k_u and k_v most reliable sub-channels
5. Save the design to `designs/gmac_{class}_n{n}_snr{snr}dB.npz`

The MC design uses a time budget to automatically stop after a specified number of seconds, making it practical for large block lengths where each trial is expensive ($O(N^2)$ for the genie-aided decoder).

3.4 Rate Selection

Given a target sum rate R_{sum} and a rate ratio $\rho = R_u/R_{\text{sum}}$, the number of information bits is:

$$k_u = \lfloor \rho \cdot R_{\text{sum}} \cdot N \rceil \tag{12}$$

$$k_v = \lfloor (1-\rho) \cdot R_{\text{sum}} \cdot N \rceil \tag{13}$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer. The actual rates $(k_u/N, k_v/N)$ are thus quantized, and the gap between target and actual rates can be significant at small N.

Additionally, k_u and k_v must not exceed the number of reliable sub-channels: $k_u \leq |\{i : Z_u(i) < \text{threshold}\}|$ and similarly for k_v. The project uses MC error rates rather than Bhattacharyya parameters for this assessment when MC designs are available.

3.5 Design File Inventory

The `designs/` directory contains pre-computed designs:

- **BEMAC:** Classes A, B, C for $n = 3, \ldots, 10$ ($N = 8, \ldots, 1024$). Additional $n = 12$ for Class B.
- **ABNMAC:** Classes A, B, C for $n = 3, \ldots, 10$.
- **Gaussian MAC:** Classes A, B, C for $n = 3, \ldots, 11$ at SNRs $\{0, 1, 2, 3, 4, 5, 6, 8, 10\}$ dB. Over 250 design files total.

4 Encoder (`encoder.py`)

The polar encoder implements $x^N = u^N \cdot G_N = u^N \cdot B_N \cdot F^{\otimes n}$ over GF(2).

4.1 Implementation

The encoding proceeds in two stages:

1. **Bit-reversal permutation:** Apply B_N by reordering the input vector according to the bit-reversal of each index. For index i with n-bit representation, compute $\text{br}[i] = \text{reverse_bits}(i, n)$, then set $x[\text{br}[i]] = u[i]$.
2. **XOR butterfly:** Apply $F^{\otimes n}$ via n stages of butterfly XOR operations. At each stage with step size $s = 1, 2, \ldots, N/2$, the array is viewed as blocks of size $2s$, and the left half of each block is XORed with the right half:

$$x_r[:, :s] \oplus= x_r[:, s:] \tag{14}$$

The complexity is $O(N \log N)$ per codeword, or $O(B \cdot N \log N)$ for batch encoding of B codewords simultaneously.

4.2 Batch and TensorFlow Variants

Three encoder variants are provided:

- `polar_encode(u)`: Single-vector encoding, returns a list.
- `polar_encode_batch(U)`: Batch NumPy encoding of shape (B, N).
- `polar_encode_batch_tf(U, N)`: TensorFlow GPU-accelerated batch encoding using `tf.bitwise.bitwise`

The helper `build_message(N, info_bits, info_positions)` places information bits at the specified 1-indexed positions, with all other positions set to their frozen values (default: 0).

5 Decoders

5.1 Overview

The project provides five decoder implementations with increasing sophistication:

Table 1: Decoder summary

Decoder	Complexity	Paths	File
Reference SC	$O(N^2)$	All	`decoder.py` (legacy)
Efficient SC	$O(N \log N)$	Extreme only	`efficient_decoder.py`
Interleaved SC	$O(N \log N)$	All monotone chain	`decoder_interleaved.py`
Parallel SC	$O(N \log N)$	Class B	`decoder_parallel.py`
SC List (SCL)	$O(L \cdot N \log N)$	All	`decoder_scl.py`

The unified public API in `decoder.py` auto-dispatches:

- Extreme paths ($\pi = 0$ or $\pi = N$) $\rightarrow$ LLR-based SC
- Class B ($\pi = N/2$) $\rightarrow$ Numba parallel decoder (if available)
- General paths $\rightarrow$ Tensor-based computational graph SC

5.2 LLR-Based SC Decoder (Extreme Paths)

For the extreme path $0^N 1^N$ (Class C), the MAC decode decomposes into:

1. Compute U-marginal leaf LLRs by marginalizing over Y:

$$\Lambda_U(t) = \ln \frac{P(z_t|x{=}0)}{P(z_t|x{=}1)} = \ln \frac{\sum_y W(z_t|0, y)}{\sum_y W(z_t|1, y)} \tag{15}$$

2. Run standard Arikan SC decoding on Λ_U to obtain $\hat{u}^N$
3. Encode $\hat{u}^N$ to get $\hat{x}^N = \hat{u}^N \cdot G_N$
4. Compute V-conditional leaf LLRs given the decoded $\hat{x}^N$:

$$\Lambda_V(t) = \ln \frac{W(z_t|\hat{x}_t, 0)}{W(z_t|\hat{x}_t, 1)} \tag{16}$$

5. Run standard SC on Λ_V to obtain $\hat{v}^N$

The Arikan SC tree uses an $O(N)$ lazy-evaluation structure (`_SCNode` class) with f-node (check) and g-node (variable) operations:

$$f(\Lambda_a, \Lambda_b) = \ln(1 + e^{\Lambda_a + \Lambda_b}) - \ln(e^{\Lambda_a} + e^{\Lambda_b}) \tag{17}$$

$$g(\Lambda_a, \Lambda_b, u) = \Lambda_b + (1 - 2u)\Lambda_a \tag{18}$$

5.3 Computational Graph Decoder (All Paths)

For intermediate paths (e.g., Class B with $\pi = N/2$), the decoder uses the computational graph approach from Ren et al. [3]. Each edge in the binary tree carries an array of 2×2 log-probability tensors $T[a, b]$ representing the joint distribution over $(X_{\text{local}}, Y_{\text{local}})$.

5.3.1 Tree Structure and Indexing

- **Edges:** Numbered $1, \ldots, 2N-1$. Edge 1 is the root. Edges $N, \ldots, 2N-1$ are leaves.
- **Vertices:** Numbered $1, \ldots, N-1$. Vertex β has parent edge β, left child edge 2β, and right child edge $2\beta + 1$.
- **Edge sizes:** Edge β at tree level ℓ carries $N/2^\ell$ tensor positions, each a 2×2 matrix.
- **Leaf mapping:** 1-indexed position i_t maps to leaf edge $i_t + N - 1$ and vertex $(i_t + N - 1) \gg 1$.

5.3.2 Core Operations

Three operations propagate information through the tree, all operating on arrays of $(L, 2, 2)$ log-probability tensors:

Circular Convolution ($\circledast$): The circular convolution marginalizes over paired positions:

$$(\circledast(A, B))[a, b] = \ln \sum_{a', b'} \exp\big(A[a \oplus a', b \oplus b'] + B[a', b']\big) \tag{19}$$

Normalized Product ($\odot$): Element-wise product (sum in log-domain) followed by normalization:

$$(A \odot B)[a, b] = A[a, b] + B[a, b] - \ln \sum_{a', b'} \exp(A[a', b'] + B[a', b']) \tag{20}$$

calcLeft, calcRight, calcParent:

- `calcLeft(`β`):` Given parent edge data and right child, compute left child:

$$E_{2\beta} = \circledast\big(E_\beta[: l],\ E_\beta[l :] \odot E_{2\beta+1}\big) \tag{21}$$

  where l is the size of the child edges.

- `calcRight(`β`):` Given parent and left child, compute right child:

$$E_{2\beta+1} = E_\beta[l :] \odot \circledast\big(E_{2\beta}, E_\beta[: l]\big) \tag{22}$$

- `calcParent(`β`):` Given both children, reconstruct parent:

$$E_\beta[: l] = \circledast(E_{2\beta}, E_{2\beta+1}), \quad E_\beta[l :] = E_{2\beta+1} \tag{23}$$

5.3.3 Navigation and decodeAt Flow

The `step_to(target)` method navigates from the current vertex to the target vertex by computing LCA paths and calling `calcLeft`, `calcRight`, or `calcParent` at each step.

The `decodeAt` procedure for each bit position i_t is:

1. Navigate to the target vertex (parent of the leaf edge)
2. Save the leaf's bottom-up data: `temp` $= E_{\text{leaf}}[0]$
3. Compute `calcLeft` or `calcRight` to get the top-down message
4. Combine: `combined` $=$ `temp` $\odot E_{\text{leaf}}[0]$
5. Make hard decision by marginalizing the combined tensor
6. Update leaf to *partially deterministic*: known dimension $\rightarrow$ delta, unknown $\rightarrow$ uniform

5.3.4 DecodeAt Algorithm

The complete algorithm for decoding a single bit position is:

Algorithm 2 DecodeAt for position i_t at path step γ (0=U, 1=V)

Require: i_t: 1-indexed position, γ: user indicator, graph state

1: leaf_edge $\leftarrow i_t + N - 1$
2: target_vertex $\leftarrow$ leaf_edge $\gg 1$
3: STEP_TO(target_vertex) ▷ Navigate tree
4: temp $\leftarrow E[\text{leaf_edge}][0]$.copy() ▷ Save bottom-up message
5: **if** leaf_edge is even **then**
6: CALCLEFT(target_vertex) ▷ Computes top-down into leaf
7: **else**
8: CALCRIGHT(target_vertex)
9: **end if**
10: combined $\leftarrow$ norm_prod($E[\text{leaf_edge}][0]$, temp)
11: **if** i_t is frozen **then**
12: $\hat{b} \leftarrow$ frozen_value
13: **else if** $\gamma = 0$ **then** ▷ U decision
14: $p_0 \leftarrow$ logsumexp(combined$[0,0]$, combined$[0,1]$)
15: $p_1 \leftarrow$ logsumexp(combined$[1,0]$, combined$[1,1]$)
16: $\hat{b} \leftarrow \mathbb{1}[p_1 > p_0]$ ▷ Marginalize over V
17: **else** ▷ V decision
18: $p_0 \leftarrow$ logsumexp(combined$[0,0]$, combined$[1,0]$)
19: $p_1 \leftarrow$ logsumexp(combined$[0,1]$, combined$[1,1]$)
20: $\hat{b} \leftarrow \mathbb{1}[p_1 > p_0]$ ▷ Marginalize over U
21: **end if**
22: Update leaf to partially deterministic tensor ▷ Eq. 16 from [3]

The partially deterministic leaf update (step 15) is critical for correctness: after deciding a bit, the leaf tensor must reflect only the known information. If both U and V values are known at position i_t, the leaf becomes a delta: $T[\hat{u}, \hat{v}] = 1$, all other entries $= 0$ (in log-domain: $T[\hat{u}, \hat{v}] = 0$, others $= -\infty$). If only one user is known, the other dimension remains uniform ($\log 0.5$).

5.3.5 Root Initialization

A critical implementation detail: the root edge entries must be bit-reversed from the leaf log-probabilities. Since the encoder applies bit-reversal before the butterfly, the tree's interleaved pairing structure requires $\text{root}[t] = \log W[\text{br}[t]]$.

Each root entry is also normalized so that the four entries sum to 1 (i.e., $\sum_{a,b} \exp(T[a,b]) = 1$).

5.4 Numba Parallel Decoder (`decoder_parallel.py`)

The parallel decoder compiles the entire $2N$-step decode loop into Numba `@njit` functions with zero Python overhead per decoded bit. Key features:

- **Flat storage:** All edge tensors are stored in a single contiguous `float64` array with precomputed offsets, avoiding Python object overhead.
- **Standalone `@njit` functions:** Each operation (`_do_cl`, `_do_cr`, `_do_cp`, `_nav`) is a standalone compiled function taking explicit array arguments. This avoids a Numba closure bug where mutable arrays in closures produce silently incorrect results.

- **Batch parallelism:** The `_decode_batch` function uses `prange` to decode multiple codewords in parallel across CPU cores.
- **Performance:** Achieves approximately 20× speedup at $N = 1024$ compared to the pure-Python computational graph decoder.

5.5 SC List Decoder (`decoder_scl.py`)

The SCL decoder maintains L candidate paths simultaneously, splitting at each information bit and pruning to keep the L most likely paths. The implementation provides:

- **Extreme path SCL:** For paths 0^N1^N and 1^N0^N, uses the Tal–Vardy log-probability propagation [4] with vectorized path operations. First decodes one user with SCL, then decodes the other user with SCL conditioned on the first.
- **General path SCL:** For intermediate paths, uses the computational graph framework with L parallel tree instances and path metric tracking.
- **Complexity:** $O(L \cdot N \log N)$ per codeword.

The SCL decoder supports list sizes $L = 1, 2, 4, 8, 16, 32$.

5.6 Batch Vectorization

All decoders support batch decoding where multiple codewords are processed in parallel via NumPy vectorization. The batched computational graph (`_CompGraphBatched`) adds a batch dimension to all edge arrays, achieving 30–40× speedup over sequential decoding.

6 Neural Decoders

6.1 Neural Computational Graph (NCG) Architecture

The neural decoder (`neural/neural_comp_graph.py`) replaces the analytical tree operations with learned MLP-based operations while preserving the $O(N \log N)$ computational graph structure.

6.1.1 Architecture Components

- **Channel Embedding:** An `nn.Embedding` layer maps discrete channel outputs $z \in \{0, 1, 2\}$ (for BEMAC) to d-dimensional vectors. Root embeddings are bit-reversed before tree processing.
- **Neural calcLeft/calcRight:** Each is a shared MLP $\mathbb{R}^{3d} \to \mathbb{R}^d$ with ELU activations and configurable depth. The input concatenates the parent's first half, parent's second half, and the sibling edge embedding.
- **Decision Head (`emb2logits`):** An MLP $\mathbb{R}^d \to \mathbb{R}^4$ producing joint logits over $(u, v) \in \{0, 1\}^2$. Shared between leaf decisions and the Soft-Bit Bridge.
- **Soft-Bit Bridge (calcParent):**
  1. Convert child embeddings to log-probabilities via `emb2logits` + temperature-scaled `log_softmax`
  2. Reshape to $(2, 2)$ tensors and apply *analytical* circular convolution (differentiable via `torch.logsumexp`)

3. Convert back to embeddings via `logits2emb` MLP ($\mathbb{R}^4 \to \mathbb{R}^d$)

This hybrid approach makes calcParent differentiable while leveraging the known algebraic structure of the circular convolution.

- **Learnable "no info" embedding:** A learnable parameter vector initializes leaf edges before they receive information.

6.1.2 Configuration

The default configuration is $d = 16$, hidden $= 64$, 2 hidden layers per MLP, temperature $\tau = 1.0$. The total parameter count is modest: typically ∼15K–30K parameters.

6.2 Memory Channel Extension (`ncg_memory.py`)

The `NCGMemoryDecoder` extends the NCG to channels with memory by replacing the discrete embedding lookup with a **Sequence Encoder**:

- A bidirectional GRU processes the continuous channel output sequence $z_1, \ldots, z_N$, producing context-aware representations.
- A projection layer maps GRU outputs to d-dimensional root embeddings.
- The tree operations remain *exactly the same* as the BEMAC model.

Two memory channel models are implemented in `channels_memory.py`:

- **ISI MAC:** $z_t = s_x(t) + s_y(t) + \alpha[s_x(t-1) + s_y(t-1)] + w_t$ with BPSK modulation, state space $S = 4$.
- **Gilbert–Elliott MAC:** 2-state Markov channel with different noise levels per state.

The key advantage is that the neural decoder's complexity is $O(md \cdot N \log N)$, independent of the channel state space size S, whereas analytical SC has complexity $O(S^3 \cdot N \log N)$.

6.3 Training Procedure (`neural/train.py`)

Training uses teacher forcing with 4-class cross-entropy loss:

1. Generate random (u, v) pairs, encode, and transmit through BEMAC
2. Run the NCG decoder forward pass with teacher-forced true values
3. At each information bit position, the model outputs 4-class logits for $(u_t, v_t) \in \{(0,0), (0,1), (1,0), (1,1)\}$
4. Compute cross-entropy loss and backpropagate

The training script supports three phases:

1. **Overfit test:** Validate architecture on 100 fixed samples
2. **Full training:** Train on Class B path with Adam optimizer and cosine annealing
3. **Evaluation:** Compare NN BLER vs. SC BLER

6.4 Saved Models

Pre-trained models are stored in `saved_models/`:

Table 2: Pre-trained NCG model inventory

File	Block Length	Notes
`ncg_N8_d16.pt`	$N = 8$	Base model
`ncg_N16_d16.pt`	$N = 16$	Base model
`ncg_N32_curriculum.pt`	$N = 32$	Curriculum learning
`ncg_N64_best.pt`	$N = 64$	Best validation
`ncg_N128_curriculum.pt`	$N = 128$	Curriculum learning
`ncg_N256_bler_sweep.pt`	$N = 256$	BLER sweep
`ncg_N512_bler_sweep.pt`	$N = 512$	BLER sweep
`ncg_N1024_bler_sweep.pt`	$N = 1024$	BLER sweep

7 Evaluation Framework

7.1 MACEval Class (`eval.py`)

The `MACEval` class provides a unified Monte Carlo evaluation pipeline:

Listing 1: Basic MACEval usage

```
from polar import BEMAC, MACEval, design_bemac, make_path

channel = BEMAC()
N, n = 1024, 10
b = make_path(N, path_i=N)
Au, Av, frozen_u, frozen_v, z_u, z_v = design_bemac(n, ku=307, kv=614)

evaluator = MACEval(channel, log_domain=True, backend='auto')
ber_u, ber_v, bler = evaluator.run(
    N, b, Au, Av, frozen_u, frozen_v, n_codewords=50000)
```

Backend selection: The `backend` parameter controls decoder dispatch:

- `'auto'`: Selects the fastest decoder for the given path (efficient for extreme paths, interleaved for others).
- `'reference'`: Forces the $O(N^2)$ reference decoder.
- `'efficient'`: Forces the efficient $O(N \log N)$ decoder (extreme paths only).
- `'interleaved'`: Forces the computational graph decoder.

Adaptive MC evaluation: The `adaptive_mc_eval` function automatically scales the number of Monte Carlo trials to fit within a given time budget by benchmarking single-decode latency first.

7.2 Simulation Scripts

The `scripts/` directory contains simulation drivers (see Section 10).

8 Results

This section presents simulation results extracted from the project's JSON result files. All BLER figures are obtained via Monte Carlo simulation with at least 50,000 codewords per data point (unless otherwise noted).

8.1 BEMAC Results: BLER vs. Block Length

8.1.1 Class C (Path $0^N 1^N$)

Table 3 shows the BLER performance for BEMAC Class C (path $0^N 1^N$, SC decoding, rate ratio $\rho = 0.6$). The BLER decreases steadily with N, confirming polarization. The V-channel BER is consistently lower than the U-channel BER because for Class C the V-conditional channel is noiseless ($Z_0^{(V)} = 0$) and all V errors come from error propagation in the U phase.

Table 3: BEMAC Class C, SC ($L = 1$), $\rho = 0.6$, 50,000 codewords, analytical design

N	k_u	k_v	R_u	R_v	BLER	BER_u
8	2	5	0.250	0.625	0.103	8.3×10^{-2}
16	5	10	0.313	0.625	0.086	4.5×10^{-2}
32	10	19	0.313	0.594	0.087	3.3×10^{-2}
64	19	38	0.297	0.594	0.057	2.0×10^{-2}
128	38	77	0.297	0.602	0.024	7.0×10^{-3}
256	77	154	0.301	0.602	0.015	3.7×10^{-3}
512	154	307	0.301	0.600	0.003	8.1×10^{-4}
1024	307	614	0.300	0.600	2.6×10^{-4}	5.2×10^{-5}

The rates converge to $(R_u, R_v) \approx (0.30, 0.60)$ as N grows. This is consistent with the target sum rate $R_u + R_v = 0.9$ at ratio $\rho = R_u/(R_u + R_v) = 1/3$. The three-order-of-magnitude drop in BLER from $N = 8$ to $N = 1024$ demonstrates clean polarization behavior.

Table 4 shows BEMAC Class A results at $\rho = 0.6$. The BLER *increases* with N, reaching 1.0 at $N \geq 512$. This confirms that the symmetric rate direction is capacity-suboptimal for the asymmetric BEMAC capacity region.

Table 4: BEMAC Class A, SC ($L = 1$), $\rho = 0.6$, design: analytical

N	k_u	k_v	$R_u = R_v$	BLER
8	4	4	0.500	0.162
16	7	7	0.438	0.273
32	14	14	0.438	0.462
64	29	29	0.453	0.764
128	58	58	0.453	0.938
256	115	115	0.449	0.997
512	230	230	0.449	1.000
1024	461	461	0.450	1.000

8.2 BEMAC Class B Results

Table 5 shows BEMAC Class B (path $0^{N/2} 1^N 0^{N/2}$). The BLER also increases with N at $\rho = 0.6$, though less dramatically than Class A. At small N (e.g., $N = 8$), Class B achieves BLER $= 0$.

8.3 Gaussian MAC Results

The Gaussian MAC campaign covers Classes A, B, C at rate ratios $\rho \in \{0.3, 0.5, 0.7, 0.9\}$ and block lengths $N = 8, \ldots, 1024$, using Monte Carlo genie-aided code design. The campaign data in `results/gmac_final_data.json` contains over 200 data points.

Table 5: BEMAC Class B, SC ($L = 1$), $\rho = 0.6$, design: analytical

N	k_u	k_v	BLER	BER_u
8	3	4	0.000	0.000
16	6	8	0.055	2.8×10^{-2}
32	12	17	0.542	0.238
64	24	34	0.762	0.323
128	48	67	0.884	0.373
256	96	134	0.986	0.411
512	192	269	1.000	0.412
1024	384	538	1.000	0.410

8.3.1 SC Decoder Results ($L = 1$)

Table 6 shows selected SC BLER results extracted from the campaign data.

Table 6: Gaussian MAC SC ($L = 1$) BLER: selected results from the campaign

Class	ρ	$N = 8$	$N = 32$	$N = 64$	$N = 128$	$N = 256$	$N = 1024$
C	0.3	0.123	0.043	0.012	0.001	0.008	0.012
C	0.5	0.109	0.076	0.027	0.007	0.002	0.002
C	0.7	0.226	0.139	0.120	0.113	0.081	0.024
C	0.9	0.198	0.326	0.492	0.573	0.652	0.774
A	0.3	0.124	0.007	0.018	0.004	0.008	0.013
A	0.5	0.046	0.008	0.003	0.000	0.001	0.000
A	0.7	0.081	0.071	0.171	0.240	0.238	0.465
A	0.9	0.684	0.955	0.999	1.000	1.000	1.000
B	0.3	0.067	0.008	0.002	0.001	0.005	0.033
B	0.5	0.038	0.007	0.007	0.000	0.000	0.001
B	0.7	0.071	0.042	0.031	0.013	0.004	0.002
B	0.9	0.249	0.505	0.650	0.726	0.892	0.998

8.3.2 SCL Decoder Results ($L = 32$)

Table 7 shows SCL results at $L = 32$ for selected configurations. The list decoder provides significant gains at moderate rates.

8.3.3 Key Observations

1. **Rate ratio sensitivity:** At moderate rate ratios ($\rho \leq 0.5$), BLER decreases with N for all classes, confirming that the codes approach capacity. At $\rho = 0.5$ for Class A, BLER $= 0.000$ is achieved already at $N = 128$.

2. **High rate ratio floor:** At $\rho = 0.9$, BLER increases with N for all classes, reaching 1.0 at large N. This indicates operation beyond the achievable rate region at the given SNR.

3. **SCL improvement:** The SCL decoder ($L = 32$) substantially improves performance at $\rho = 0.7$. For example, Class B at $N = 1024$ goes from BLER $= 0.002$ (SC) to 0.000 (SCL). At Class C with $\rho = 0.7$, SCL achieves BLER $= 0$ at $N = 1024$ compared to SC's 0.024.

Table 7: Gaussian MAC SCL ($L = 32$) BLER: selected results

Class	ρ	$N = 8$	$N = 32$	$N = 64$	$N = 128$	$N = 256$	$N = 1024$
C	0.5	0.116	0.038	0.009	0.000	0.000	0.000
C	0.7	0.222	0.148	0.087	0.031	0.016	0.000
A	0.5	0.033	0.004	0.004	0.000	0.000	0.000
A	0.7	0.044	0.056	0.081	0.141	0.108	0.180
B	0.5	0.032	0.003	0.004	0.000	0.000	0.000
B	0.7	0.036	0.025	0.009	0.004	0.002	0.000

4. **Class comparison:** Class B with $\rho = 0.7$ scales better than Class A with the same ρ. At $N = 1024$, Class B achieves BLER $= 0.002$ while Class A has 0.465 under SC.

5. **Non-monotone BLER:** Some configurations show non-monotone BLER vs. N behavior (e.g., Class A at $\rho = 0.3$ with BLER dipping to 0.004 at $N = 128$ then rising to 0.013 at $N = 1024$). This is attributed to the discrete rate quantization: the actual rates $(k_u/N, k_v/N)$ cannot exactly match the target at every N.

8.4 ABNMAC Results

ABNMAC simulations span Classes A, B, C with MC-based design at block lengths $N = 8$ to $N = 1024$. The ABNMAC channel has asymmetric capacities ($I(Z;X) \approx 0.400$, $I(Z;Y|X) \approx 0.800$), making it more challenging than BEMAC.

At $N = 8$, Class C achieves BLER ≈ 0.119 with $k_u = 2$, $k_v = 3$, using the $O(N \log N)$ efficient decoder with MC design. The ABNMAC requires the Monte Carlo design procedure since the Bhattacharyya parameters depend on the full noise distribution.

8.4.1 ABNMAC Design Considerations

A key difference from BEMAC is that neither user has a noiseless conditional channel. The V-conditional Bhattacharyya starting value is $Z_0^{(V)} \approx 0.337$ (compared to 0 for BEMAC), meaning the V-channel also experiences polarization and not all V positions can carry information. This reduces the achievable rates compared to the capacity bounds, particularly at short block lengths.

8.5 SC vs. SCL Comparison on BEMAC

To quantify the benefit of list decoding, we compare SC ($L = 1$) and SCL ($L = 32$) on BEMAC for various rate ratios. The BEMAC is an ideal testbed because the V-conditional channel is noiseless for Class C, isolating the impact of list decoding on the U-phase.

At aggressive rate ratios ($\rho = 0.6$–0.8), SCL provides notable improvements over SC, particularly at intermediate block lengths ($N = 32$–256) where the finite-length penalty is most pronounced. At $\rho = 0.5$ (the most conservative setting), both SC and SCL achieve very low BLER, making the list decoder's advantage less visible.

8.6 Results Directory Structure

The `results/` directory is organized as follows:

- **JSON files:** Primary simulation data, named by convention `sim_{channel}_{class}_{design}_L{list_s`
  Each file contains metadata (channel, class, decoder type, design method) and a list of result records with N, k_u, k_v, BLER, BER$_u$, BER$_v$, block error count, codeword count, and wall-clock time.

- **Campaign files:** `campaign_10h.json` and `campaign_28h.json` contain multi-day Gaussian MAC simulation campaigns with hundreds of data points.
- **Plot files:** PNG and PDF figures generated by the plotting scripts, organized into `final_plots/` (publication-quality) and `clean_plots/` (intermediate) subdirectories.
- **NCG results:** `ncg_vs_sc_classB.json` and `nn_vs_sc_bemac_*.json` contain neural decoder evaluation results.

8.7 Generated Plots

The plotting scripts generate the following figure types:

1. **BLER vs. N by class:** Compares Classes A, B, C at fixed SNR and rate ratio (e.g., `01_classes_snr3dB.png`).
2. **BLER vs. N by rate ratio:** Compares $\rho = 0.3, 0.5, 0.7, 0.9$ for a fixed class (e.g., `02_rho_snr6dB.png`).
3. **SC vs. SCL comparison:** Overlay SC and SCL curves (e.g., `03_scl_snr3dB.png`).
4. **Rate vs. BLER:** BLER as a function of sum rate at fixed N (e.g., `04_rate_snr6dB.png`).
5. **SNR sweep:** BLER vs. SNR for fixed N and class (e.g., `05_snr_compare.png`).
6. **Heatmap:** BLER as a function of (N, ρ) for a given class and SNR (e.g., `05_heatmap_clean.png`).

8.8 NCG Neural Decoder Results

Table 8 compares the NCG neural decoder against analytical SC on BEMAC Class B.

Table 8: NCG vs. SC BLER on BEMAC Class B

N	k_u	k_v	NN BLER	SC BLER
8	4	6	0.0586	0.0582
16	8	11	0.0106	0.0118
32	16	22	0.0082	0.0096
64	32	45	0.0044	0.0056
128	64	90	0.0018	0.0034

The neural decoder matches SC at $N = 8$ and progressively outperforms it as N increases. At $N = 128$, the NCG achieves a 47% reduction in BLER (0.0018 vs. 0.0034). This is achieved with curriculum learning and a modest model size ($d = 16$, hidden $= 64$).

9 Implementation Notes

This section documents key implementation details and design decisions that are important for understanding and extending the codebase.

9.1 Indexing Conventions

The project consistently uses **1-based indexing** for polar code positions:

- Information sets A_U, A_V contain 1-indexed positions
- Frozen dictionaries map 1-indexed positions to frozen values

- The decoder's internal coordinate probabilities use 1-indexed (i, j)
- The computational graph uses 1-indexed edges $(1, \ldots, 2N-1)$ and vertices $(1, \ldots, N-1)$

Internal arrays (e.g., Bhattacharyya parameters, error rates) use 0-indexed NumPy arrays. The conversion between conventions is consistently handled at the interface boundaries.

9.2 Log-Domain Arithmetic

All probability computations are performed in the log-domain to avoid numerical underflow. The key operations are:

- `np.logaddexp(a, b)`: Computes $\log(e^a + e^b)$ stably
- Products become sums: $\log(p \cdot q) = \log p + \log q$
- Normalization: subtract $\log \sum_i e^{x_i}$ from all entries
- Zero probability: represented as $-\infty$ (`-np.inf`)

The circular convolution involves a 4-term `logaddexp` for each of the 4 output entries, totaling 16 additions and 12 `logaddexp` operations per tensor pair. This is the computational bottleneck of the decoder.

9.3 Numerical Stability

Special care is taken for edge cases:

- **NaN handling in LLR f-node:** When both Λ_a and Λ_b are $\pm\infty$, the `logaddexp` formula can produce NaN. The implementation falls back to the min-sum approximation: $f(\Lambda_a, \Lambda_b) \approx \text{sign}(\Lambda_a) \cdot \text{sign}(\Lambda_b) \cdot \min(|\Lambda_a|, |\Lambda_b|)$.
- **Normalization of all-$-\infty$ tensors:** The `norm_prod` function checks whether the total is finite before subtracting; if not, the tensor is left unnormalized.
- **BEMAC deterministic transitions:** For BEMAC, the leaf log-probabilities are exactly 0 or $-\infty$, making the arithmetic exact (no floating-point approximation).

9.4 Performance Optimization

The project employs several levels of optimization:

1. **Vectorized batch encoding:** The `polar_encode_batch` function processes all codewords simultaneously using NumPy array operations, with the butterfly XOR stage using reshape + slice + XOR.
2. **Batched SC decoding:** The `_CompGraphBatched` class adds a batch dimension to all edge arrays. All batch elements share the same tree traversal pattern (since they use the same path b), so the control flow is identical—only the data differs. This achieves 30–40× speedup over sequential decoding.
3. **Numba JIT compilation:** The parallel decoder (`decoder_parallel.py`) compiles all inner loops to native code via Numba's `@njit` decorator. The flat array storage avoids Python object overhead. The `prange` directive enables automatic parallelization across CPU cores for batch decoding.
4. **Lazy evaluation:** The `_SCNode` tree only computes LLRs when requested (`get_llr(k)`), and the computational graph only evaluates edges when navigated to (`step_to`).

9.5 Numba Closure Bug

An important lesson from the parallel decoder development: Numba closures that capture mutable arrays (e.g., the edge data array) can produce *silently incorrect results* because Numba may not properly track mutations to captured arrays. The solution is to make all `@njit` functions *standalone*, taking the mutable array as an explicit argument rather than capturing it from a closure scope.

9.6 Design Comparison: Analytical vs. Monte Carlo

The project provides a `compare_with_analytical` utility that runs both the analytical Bhattacharyya design and the MC design on the same channel and reports whether the resulting information sets agree. For BEMAC with the extreme path $0^N 1^N$, the two methods produce identical designs at all tested block lengths, validating both implementations.

For ABNMAC and Gaussian MAC, the MC design is the primary method since the analytical approach requires closed-form starting parameters that may not capture the full complexity of intermediate paths.

10 Scripts Reference

Script	Description
`scripts/simulate.py`	General-purpose SC/SCL simulator for BEMAC
`scripts/simulate_abnmac.py`	ABNMAC-specific SC simulator
`scripts/simulate_gmac.py`	Gaussian MAC simulator with SNR parameter
`scripts/bler_vs_N.py`	Sweep BLER vs. N for a given channel and class
`scripts/bler_vs_N_bemac.py`	BEMAC-specific BLER vs. N sweep with analytical design
`scripts/run_bler_vs_N.py`	Batch runner for BLER vs. N across classes
`scripts/run_design.py`	Run MC design for BEMAC and save to `designs/`
`scripts/run_design_abnmac.py`	Run MC design for ABNMAC
`scripts/run_design_gmac.py`	Run MC design for Gaussian MAC
`scripts/campaign_10h.py`	10-hour Gaussian MAC simulation campaign
`scripts/campaign_28h.py`	28-hour extended campaign
`scripts/overnight_gmac.py`	Overnight Gaussian MAC campaign
`scripts/benchmark_parallel.py`	Benchmark Numba parallel decoder speedup
`scripts/demo_one_codeword.py`	Single codeword encode/decode demonstration
`scripts/plot_results.py`	General BEMAC result plotter
`scripts/plot_abnmac.py`	ABNMAC result plotter
`scripts/plot_bler_vs_N_all_classes.py`	Multi-class BLER vs. N comparison plots
`scripts/plot_gmac_final.py`	Gaussian MAC final publication-quality plots
`scripts/generate_decoder_pdf.py`	Generate decoder architecture diagrams

11 File Structure and API Reference

11.1 Project File Tree

```
to_git_v2/
  polar/                          # Core library
    __init__.py                   # Public API exports
    channels.py                   # BEMAC, ABNMAC, GaussianMAC
```

```
    encoder.py                  # Polar encoder (BR + butterfly)
    decoder.py                  # Unified SC decoder (auto-dispatch)
    decoder_scl.py              # SC List decoder
    decoder_interleaved.py      # O(N log N) comp. graph decoder
    decoder_parallel.py         # Numba JIT parallel decoder
    efficient_decoder.py        # O(N log N) LLR decoder (extreme)
    _decoder_numba.py           # Low-level Numba helpers
    design.py                   # Analytical Bhattacharyya design
    design_mc.py                # Monte Carlo genie-aided design
    eval.py                     # MACEval evaluation pipeline

  neural/                       # Neural decoder
    __init__.py
    neural_comp_graph.py        # NCG decoder (MLP + Soft-Bit Bridge)
    ncg_memory.py               # NCG with GRU sequence encoder
    channels_memory.py          # ISI MAC, Gilbert-Elliott MAC
    train.py                    # Training script

  designs/                      # Pre-computed design files (.npz)
  saved_models/                 # Pre-trained NCG models (.pt)
  results/                      # Simulation results (.json, .png)
    final_plots/                # Publication-quality figures
    clean_plots/                # Cleaned figures
  scripts/                      # Simulation and plotting scripts
  docs/                         # Documentation
  design.py                     # Top-level design entry point
```

11.2 Public API (`polar/__init__.py`)

The public API exports the following:

Table 10: Public API

Module	Symbols	Purpose
`encoder`	`polar_encode`, `build_message`	Encoding
`channels`	`BEMAC`, `ABNMAC`, `GaussianMAC`	Channels
`design`	`design_bemac`, `make_path`	Design
`design_mc`	`design_bemac_mc`	MC design
`decoder`	`decode_single`, `decode_batch`	SC decoding
`decoder_scl`	`decode_single_list`, `decode_batch_list`	SCL decoding

Additional internal modules:

- `design_mc.design_from_file()`: Load MC design from `.npz` and build info/frozen sets.
- `eval.MACEval`: Full Monte Carlo evaluation pipeline with backend selection.
- `eval.adaptive_mc_eval()`: Time-budgeted evaluation.

12 References

References

[1] E. Arikan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inform. Theory*, vol. 55, no. 7, pp. 3051–3073, July 2009.

[2] S. Önay, "Successive cancellation decoding of polar codes for the two-user binary-input MAC," in *Proc. IEEE Int. Symp. Inform. Theory (ISIT)*, Istanbul, Turkey, July 2013, pp. 1368–1372.

[3] Y. Ren, R. Nasser, and U. Bhatt, "Polar codes for the multiple access channel via monotone chain decoding," 2025.

[4] I. Tal and A. Vardy, "How to construct polar codes," *IEEE Trans. Inform. Theory*, vol. 59, no. 10, pp. 6562–6582, Oct. 2013.